Windows Systems and Artifacts

Windows 10 Artifacts

The Windows Operating System

P1KA-S4UR

Version	1.0
Date of Version	2023 June 01
Created by	Jerome Gutierrez
Confidentiality Level	Internal

Overview

1. Windows OS History
2. Windows OS Architecture
3. Windows File System
4. Windows Operations
5. Registry Entries location
6. Event Log Location
7. Prefetch Files
8. Windows OS Executable File

Windows History

MS-DOS | OS Versions | OS Vulnerability

Disk Operating System (DOS)

- The Disk Operating System (DOS) is an operating system that the computer uses to enable the data storage devices to read and write files
- DOS provides a file system which organizes the files in a specific way on the disk
- MS-DOS, created by Microsoft, used a command line as the interface for people to create programs and manipulate data files. DOS commands are shown in bold text in the given command output
- With MS-DOS, the computer had a basic working knowledge of accessing the disk drive and loading the operating system files directly from disk as part of the boot process

```
Starting MS-DOS...
HIMEM is testing extended memory... done.
C:\> C:\DOS\SMARTDRV.EXE /X
C:\> dir
Volume in drive C is MS-DOS_6
Volume Serial Number is 4006-6939
Directory of C:\
DOS          <DIR>          05-06-17  1:09p
COMMAND  COM       54,645 05-31-94  6:22a
WINA20   386        9,349 05-31-94  6:22a
CONFIG   SYS           71 05-06-17  1:10p
AUTOEXEC BAT           78 05-06-17  1:10p
        5 file(s)        64,143 bytes
                   517,021,696 bytes free
C:\>
```

Disk Operating System (DOS)

- Early versions of Windows consisted of a Graphical User Interface (GUI) that ran over MS-DOS, starting with Windows 1.0 in 1985
- In newer versions of Windows, built on New Technologies (NT), the operating system itself is in direct control of the computer and its Hardware
- Today, many things that used to be accomplished through the command line interface of MS-DOS can be accomplished in the Windows GUI
- To experience a little of MS-DOS, open a command window by typing **cmd** in Windows Search and pressing **Enter**.

Disk Operating System (DOS)

The following table lists some of the commands of MS-DOS:

MS-DOS Command	Description
dir	Show a listing of all files in the current directory (folder)
cd directory	Changes directory to the indicated directory
cd ..	Changes directory to the directory above the current directory
cd \	Changes directory to the root directory (often C)
copy source destination	Copies files to another location
del	Deletes one or more files
find	Searches for text in files
mkdir directory	Creates new Directory
ren oldname newname	Renames a file
help	Displays all the commands that can be used, with a brief description

Windows Versions

- Since 1993, there have been more than 20 releases of Windows that are based on the NT operating system (OS).
- Many editions were built specifically for workstation, professional, server, advanced server, and datacenter server, to name just a few of the many purpose-built versions.
- The 64-bit operating system was an entirely new architecture. It had a 64-bit address space instead of a 32-bit address space

Windows Versions

- 64-bit computers and operating systems are backward-compatible with older, 32-bit programs, but 64-bit programs cannot be run on older, 32-bit hardware

- With each subsequent release of Windows, the operating system has become more refined by incorporating more features

- Microsoft has announced that Windows 10 is the last version of Windows. Rather than purchasing new operating systems, users will just update Windows 10 instead.*

Windows Versions

The following table lists some of the commands of MS-DOS:

OS	Versions
Windows 7	Starter, Home Basic, Home premium, Professional, Enterprise, Ultimate
Windows Server 2008 R2	Foundation, Standard, Enterprise, Datacenter, Web Server, HPC Server Itanium-Based Systems
Windows Home server 2011	None
Windows 8	Windows 8, Windows 8 Pro, Windows 8 Enterprise, Windows RT
Windows Server 2012	Foundation, Essentials, Standard, Datacenter
Windows 8.1	Windows 8.1, Windows 8.1 Pro. Windows 8.1 Enterprise. Windows RT 8.1
Windows Server 2012 R2	Foundation, Essentials, Standard, Datacenter
Windows 10	Home, Pro, Pro Education, Enterprise, Education, IoT Core, Mobile, Mobile Enterprise
Windows Server 2016	Essentials, Standard, Datacenter, Multipoint Premium Server, Storage Server, Hyper-V Server

Operating System Vulnerabilities

- Operating systems consist of millions of lines of code. With all this code comes vulnerabilities

- A vulnerability is some flaw or weakness that can be exploited by an attacker to directly affect or reduce the viability of a computer's Information

- To take advantage of an operating system vulnerability, the attacker must use a technique or a tool to exploit the vulnerability.

Operating System Vulnerabilities

The following table lists some common Windows OS Security recommendations:

Recommendation	Description
Virus or malware protection	• By default, Windows uses Windows Defender for malware protection. • Windows Defender provides a suite of protection tools built into the system • If windows defender is turned off, the system becomes more vulnerable to attacks and malware
Unknown or unmanaged services	• There are many services that run behind the scene • It is important to make sure that each service is identifiable and safe • With an unknown service running in the background, the computer can be vulnerable to attack
Encryption	• When data is not encrypted, it can be gathered and exploited • This is not only important for desktop computers, but especially mobile devices
Security Policy	• A good security policy must be configured and followed • Many settings in the Windows Security Policy can prevent attacks

Operating System Vulnerabilities

The following table lists some common Windows OS Security recommendations:

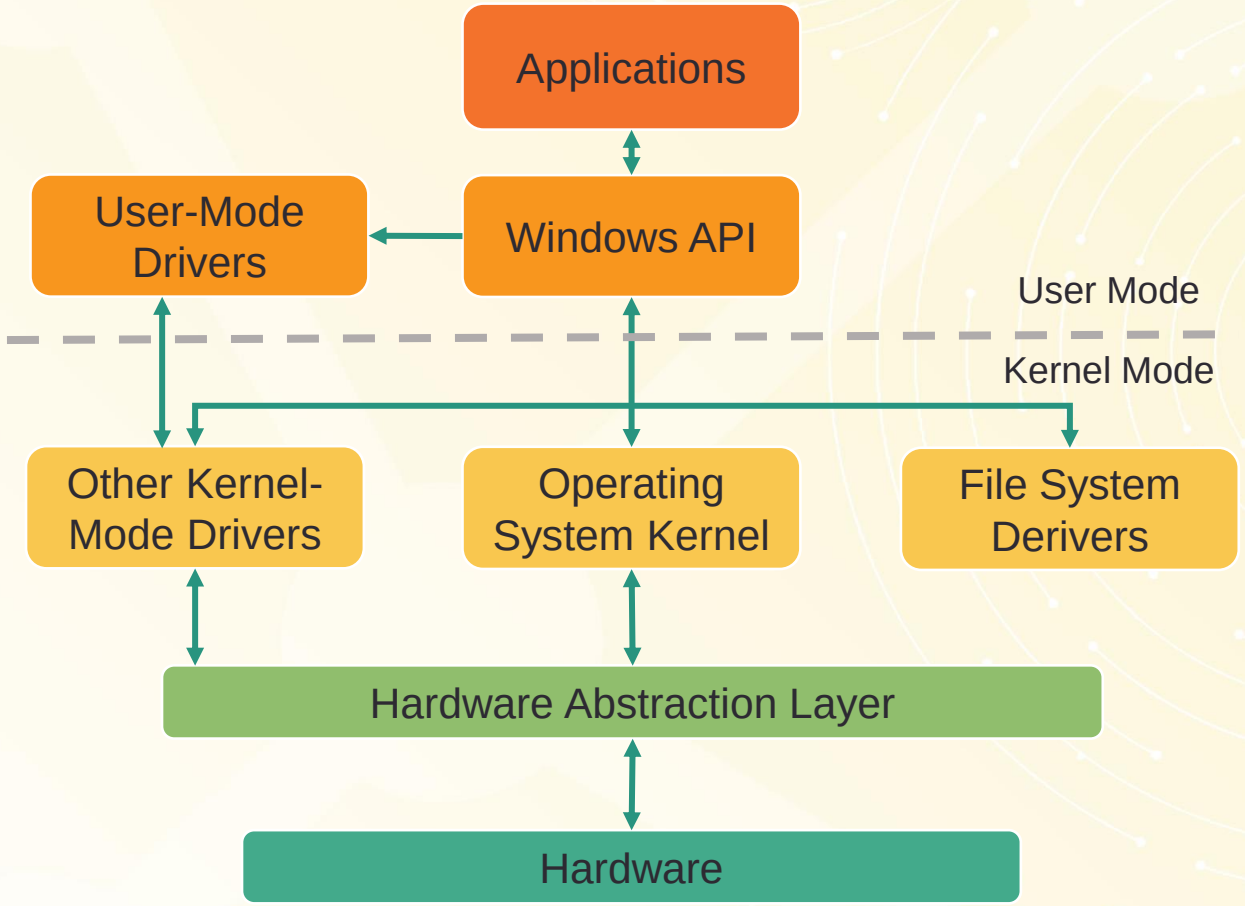
Recommendation	Description
Firewall	• By default, Windows Defender uses Windows Firewall to limit communication with devices on the network Over time, rules may no longer apply • It is important to **review firewall settings periodically** to ensure that the rules are applicable and remove any that no longer apply.
File and share permissions	• These permissions must be set correctly. It is easy to give the "Everyone" group Full Control, but this allows all people to access any files. • It is best to provide each user or group with the minimum permissions all files and folders.
Weak or no password	• Many people choose weak passwords or do not use a password at all • It is especially important to make sure that all accounts, especially the Administrator account have a very strong password.
Login as Administrator	• When a user logs in as an administrator, any program that they run will have the privileges of that account • It is best to log in as a Standard User and only use the administrator password to accomplish certain tasks

Windows Architecture

Hardware Abstraction Layer | File Systems →FAT→NTFS

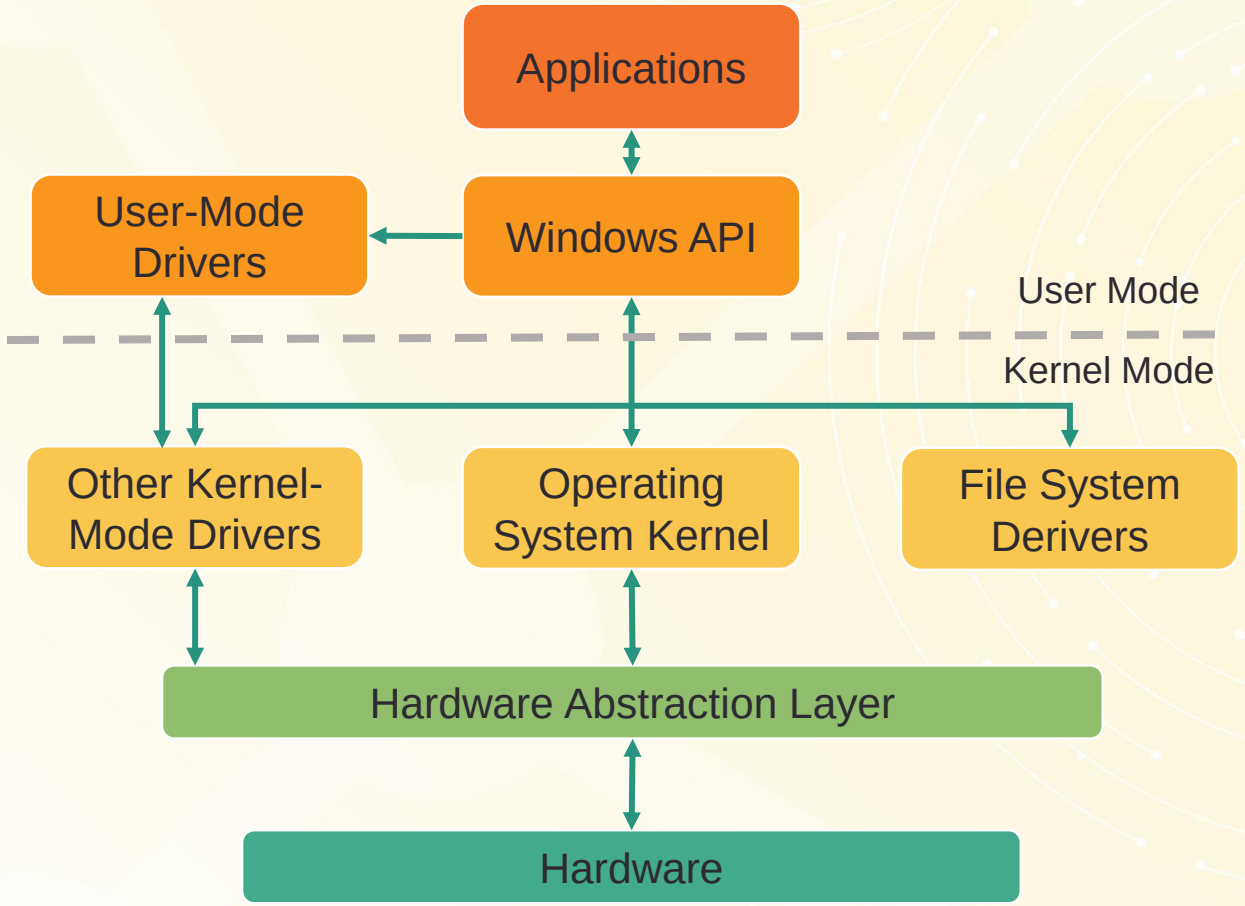
Hardware Abstraction Layer (HAL)

- A hardware abstraction layer (HAL) is software that handles all of the communication between the hardware and the kernel
- The kernel is the core of the operating system and has control over the entire computer
- The kernel handles all of the input and output request, memory, and all of the peripherals connected to the computer
- The basic Windows architecture is shown in the figure



Hardware Abstraction Layer (HAL)

- The two different modes in which a CPU operates when the computer has Windows installed are the user mode and the kernel mode
- Installed applications run in user mode, and operating system code runs in kernel mode
- All of the code that runs in kernel mode uses the same address space
- When user mode code runs, it is granted its own restricted address space by the kernel, along with a process created specifically for the application



Windows File Systems

File Systems →FAT→NTFS

Windows File Systems

A file system is a way of organizing the information on storage media. The following table lists the file systems supported by Windows:

Windows File System	Description
exFAT	• This is a simple file system supported by many different operating systems. • FAT has limitations to the number Of partitions, partition sizes, and file sizes that it can address, so, it is not usually used for hard drives or solid-state drives anymore • Both FAT 16 and FAT32 are available to use. with FAT32 being the most common as It has many fewer restrictions than FAT 16.
Hierarchical File System Plus (HFS+)	• This file system is used on MAC OS X computers and much longer filenames file sizes, and partition sizes • Although it not supported by Windows without special software, Windows is able to read data from HFS+ partitions.

Windows File Systems

A file system is a way of organizing the information on storage media. The following table lists the file systems supported by Windows:

Windows File System	Description
Extended File system (EXT)	• This file system is used with Linux-based computers • Although it not supported by Windows, Windows is able to read data from EXT partitions WITH special software.
New Technology File system (NTFS)	• This is the most commonly used file system when installing Windows. All versions of Windows and Linux Support NTFS. • Has ability to set access control lists (ACLs, or permissions) • Mac-OS X computers can read an NTFS partition. They are able to write to an NTFS partition after installing special drivers.

File Allocation Table (FAT FS)

- Primary file system on MS-DOS
- Used extensively in Windows 3.x,Windows 95,Windows 98
- FAT12, FAT16, FAT32, exFAT (differ in cluster size)
- Composed of:
  - boot sector
  - FAT areas 1 and 2
  - root folder
  - Files
  - other folders

File Allocation Table (FAT FS)

Boot sector (first sector)

```
0000000   eb  3f  90  49  42  4d  20  20  33  2e  33  00  02  01  01  00
0000010   02  e0  00  40  0b  f0  09  00  12  00  02  00  00  00  00  00
0000020   00  00  00  00  00  00  00  00  00  00  70  00  ff  ff  49  42
0000030   4d  42  49  4f  20  20  43  4f  4d  00  50  00  00  08  00  18
...
```

Byte #	Content	Value
11-12	Number of bytes per sector	0x0200 (512)
13	Number of sectors per cluster	0x01 (1)
16	Number of FAT copies	0x02 (2)
17	Number of root directory entries	0xe0 (224)

File Allocation Table (FAT FS)

Clusters are referenced by:

- 32-byte entries that contain the address of the starting cluster
- if a file consumes more than one cluster, then the starting cluster ends with the number of the next cluster and so on until the last cluster is reached
- final cluster is marked with an end-of-file indicator (0xffff).

New Technology File System (NTFS)

NTFS formatting creates important structures on the disk for file storage, and tables for recording the locations of files:

- **System Files**: These are hidden files that store information about other

volumes and file attributes

- **File Area:** The main area of the partition where files and directories are Stored

Note: When formatting a partition, the **previous data may still be recoverable** because not all the data is completely removed. It is recommended to **perform a secure wipe** on a drive that is being reused. The secure wipe will write data to the entire drive multiple times **to ensure there is no remaining data.**

New Technology File System (NTFS)

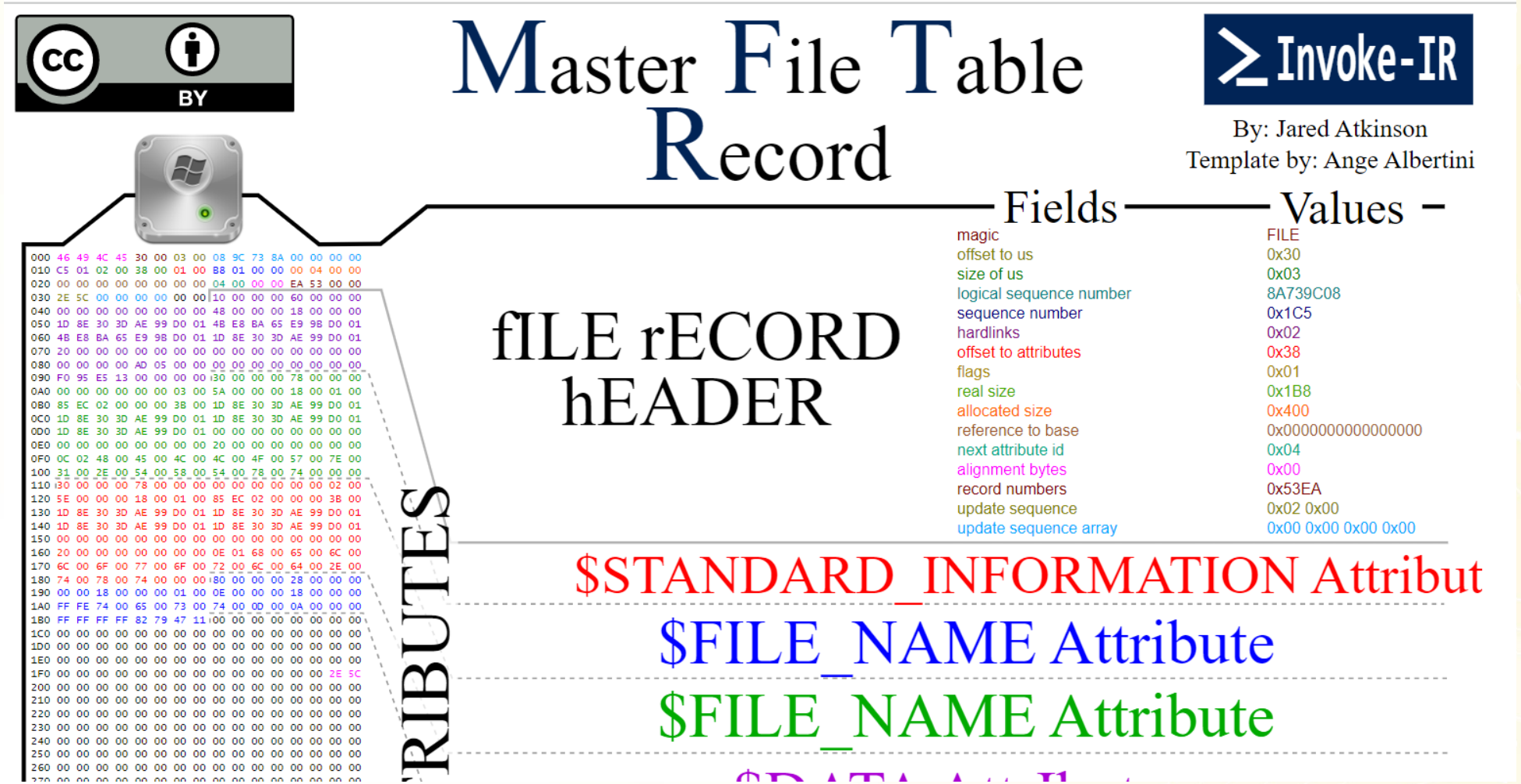
NTFS formatting creates important structures on the disk for file storage, and tables for recording the locations of files:

- **Partition Boot Sector:** This is the **first 16 sectors** of the drive. It contains the location of the Master File Table (MFT).The last 16 sectors contain a copy of the boot sector.
- **Master File Table (MFT):** This table contains the locations of all the **files and directories on the partition**, including file attributes such as security information and timestamps. Each MFT entry is 1024 bytes

MFT Entry Header	**Attribute**	**Attribute**	**Attribute**	**Unused space**

New Technology File System (NTFS)

Master File Table (MFT) Entry



New Technology File System (NTFS)

Master File Table (MFT) Entry

First 42 bytes comprise a header structure with 12 elements

```
00000000h: 46 49 4C 45 30 00 03 00 64 6F 11 01 00 00 00 00 ; FILE0...do......
00000010h: 01 00 01 00 38 00 01 00 A0 01 00 00 00 04 00 00 ; ....8...........
00000020h: 00 00 00 00 00 00 00 00 06 00 00 00 00 00 00 00 ; ................
00000030h: 50 00 00 00 00 00 00 00 10 00 00 00 60 00 00 00 ; P...........`...
00000040h: 00 00 18 00 00 00 00 00 48 00 00 00 18 00 00 00 ; ........H.......
00000050h: 26 B8 8D B1 12 55 C4 01 26 B8 8D B1 12 55 C4 01 ; &¸.±.UÄ.&¸.±.UÄ.
00000060h: 26 B8 8D B1 12 55 C4 01 26 B8 8D B1 12 55 C4 01 ; &¸.±.UÄ.&¸.±.UÄ.
00000070h: 06 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ; ................
00000080h: 00 00 00 00 00 01 00 00 00 00 00 00 00 00 00 00 ; ................
00000090h: 00 00 00 00 00 00 00 00 30 00 00 00 68 00 00 00 ; ........0...h...
```

New Technology File System (NTFS)

Master File Table (MFT) Entry

First 42 bytes comprise a header structure with 12 elements

Signature

Link count

flags

```
00000000h: 46 49 4C 45 30 00 03 00 64 6F 11 01 00 00 00 00 ; FILE0...do......
00000010h: 01 00 01 00 38 00 01 00 A0 01 00 00 00 04 00 00 ; ....8...........
00000020h: 00 00 00 00 00 00 00 00 06 00 00 00 00 00 00 00 ; ................
00000030h: 50 00 00 00 00 00 00 00 10 00 00 00 60 00 00 00 ; P...........`...
00000040h: 00 00 18 00 00 00 00 00 48 00 00 00 18 00 00 00 ; ........H.......
00000050h: 26 B8 8D B1 12 55 C4 01 26 B8 8D B1 12 55 C4 01 ; &¸.±.UÄ.&¸.±.UÄ.
00000060h: 26 B8 8D B1 12 55 C4 01 26 B8 8D B1 12 55 C4 01 ; &¸.±.UÄ.&¸.±.UÄ.
00000070h: 06 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ; ................
00000080h: 00 00 00 00 00 01 00 00 00 00 00 00 00 00 00 00 ; ................
00000090h: 00 00 00 00 00 00 00 00 30 00 00 00 68 00 00 00 ; ........0...h...
```

Sequence number

Attribute offset

New Technology File System (NTFS)

Master File Table (MFT) Entry

Remaining 982 bytes depend largely on the values within the header

```
00000000h:  46  49  4C  45  30  00  03  00  64  6F  11  01  00  00  00  00  ; FILE0...do......
00000010h:  01  00  01  00  38  00  01  00  A0  01  00  00  00  04  00  00  ; ....8...........
00000020h:  00  00  00  00  00  00  00  00  06  00  00  00  00  00  00  00  ; ................
00000030h:  50  00  00  00  00  00  00  00  10  00  00  00  60  00  00  00  ; P...........`...
00000040h:  00  00  18  00  00  00  00  00  48  00  00  00  18  00  00  00  ; ........H.......
00000050h:  26  B8  8D  B1  12  55  C4  01  26  B8  8D  B1  12  55  C4  01  ; &¸.±.UÄ.&¸.±.UÄ.
00000060h:  26  B8  8D  B1  12  55  C4  01  26  B8  8D  B1  12  55  C4  01  ; &¸.±.UÄ.&¸.±.UÄ.
00000070h:  06  00  00  00  00  00  00  00  00  00  00  00  00  00  00  00  ; ................
00000080h:  00  00  00  00  00  01  00  00  00  00  00  00  00  00  00  00  ; ................
00000090h:  00  00  00  00  00  00  00  00  30  00  00  00  68  00  00  00  ; ........0...h...
```

New Technology File System (NTFS)

Master File Table (MFT) Entry

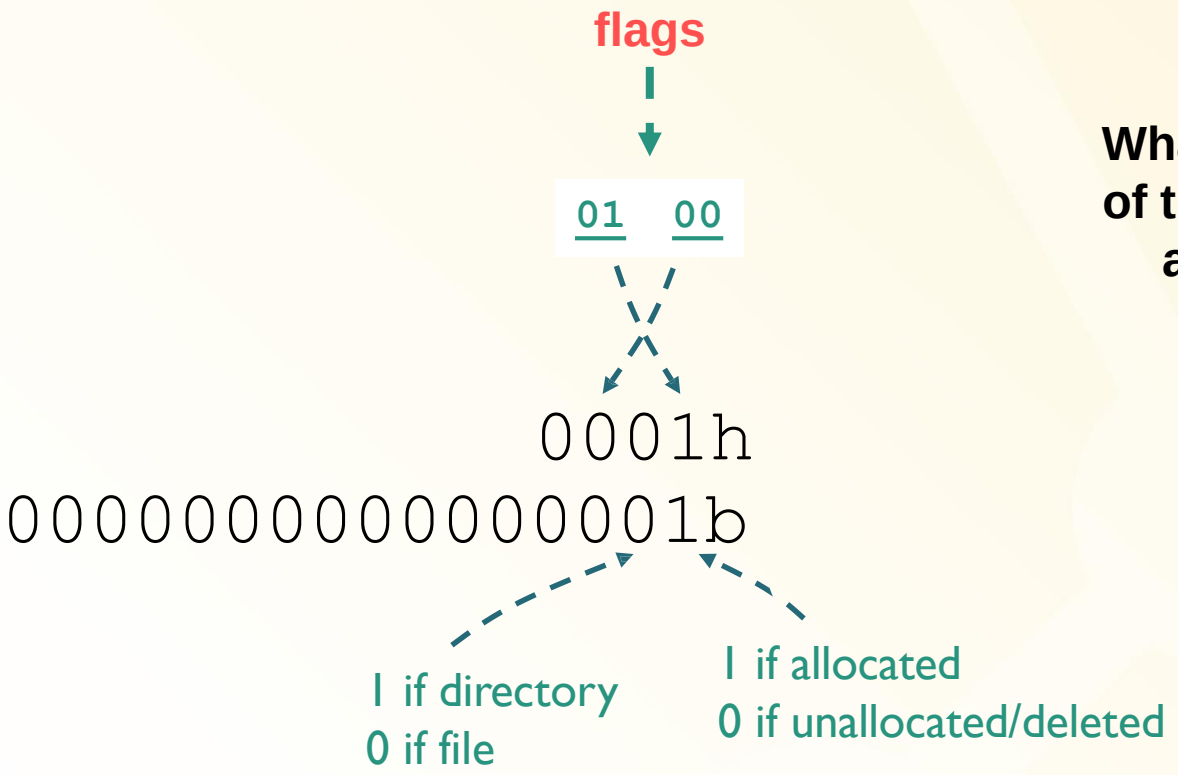
Flags determine entry type and allocation status

flags

```
00000000h:  46  49  4C  45  30  00  03  00  64  6F  11  01  00  00  00  00  ; FILE0...do......
00000010h:  01  00  01  00  38  00  01  00  A0  01  00  00  00  04  00  00  ; ....8...........
00000020h:  00  00  00  00  00  00  00  00  06  00  00  00  00  00  00  00  ; ................
00000030h:  50  00  00  00  00  00  00  00  10  00  00  00  60  00  00  00  ; P...........`...
00000040h:  00  00  18  00  00  00  00  00  48  00  00  00  18  00  00  00  ; ........H.......
00000050h:  26  B8  8D  B1  12  55  C4  01  26  B8  8D  B1  12  55  C4  01  ; &¸.±.UÄ.&¸.±.UÄ.
00000060h:  26  B8  8D  B1  12  55  C4  01  26  B8  8D  B1  12  55  C4  01  ; &¸.±.UÄ.&¸.±.UÄ.
00000070h:  06  00  00  00  00  00  00  00  00  00  00  00  00  00  00  00  ; ................
00000080h:  00  00  00  00  00  01  00  00  00  00  00  00  00  00  00  00  ; ................
00000090h:  00  00  00  00  00  00  00  00  30  00  00  00  68  00  00  00  ; ........0...h...
```

NTFS Master File Table (MFT) Entry

Flags determine entry type and allocation status



What should be the value of the flags to indicate an allocated directory?

0003h

NTFS Master File Table (MFT) Entry

Attribute offset points to the start of the first file/folder attribute

Attribute offset

First Attribute:
Standard Information Block

```
00000000h:  46  49  4C  45  30  00  03  00  64  6F  11  01  00  00  00  00  ; FILE0...do......
00000010h:  01  00  01  00  38  00  01  00  A0  01  00  00  00  04  00  00  ; ....8...........
00000020h:  00  00  00  00  00  00  00  00  06  00  00  00  00  00  00  00  ; ................
00000030h:  50  00  00  00  00  00  00  00  10  00  00  00  60  00  00  00  ; P...........`...
00000040h:  00  00  18  00  00  00  00  00  48  00  00  00  18  00  00  00  ; ........H.......
00000050h:  26  B8  8D  B1  12  55  C4  01  26  B8  8D  B1  12  55  C4  01  ; &¸.±.UÄ.&¸.±.UÄ.
00000060h:  26  B8  8D  B1  12  55  C4  01  26  B8  8D  B1  12  55  C4  01  ; &¸.±.UÄ.&¸.±.UÄ.
00000070h:  06  00  00  00  00  00  00  00  00  00  00  00  00  00  00  00  ; ................
00000080h:  00  00  00  00  00  01  00  00  00  00  00  00  00  00  00  00  ; ................
00000090h:  00  00  00  00  00  00  00  00  30  00  00  00  68  00  00  00  ; ........0...h...
```

NTFS Master File Table (MFT) Entry

Attribute header has a <u>standard format for first 16 bytes</u>

First 4 bytes is the **attribute ID** ($type), e.g.:

0x10 = $STANDARD_INFORMATION
0x30 = $FILE_NAME
0x80 = $DATA

Attribute Header

```
00000000h:  46  49  4C  45  30  00  03  00  64  6F  11  01  00  00  00  00  ; FILE0...do......
00000010h:  01  00  01  00  38  00  01  00  A0  01  00  00  00  04  00  00  ; ....8...........
00000020h:  00  00  00  00  00  00  00  00  06  00  00  00  00  00  00  00  ; ................
00000030h:  50  00  00  00  00  00  00  00  10  00  00  00  60  00  00  00  ; P...........`...
00000040h:  00  00  18  00  00  00  00  00  48  00  00  00  18  00  00  00  ; ........H.......
00000050h:  26  B8  8D  B1  12  55  C4  01  26  B8  8D  B1  12  55  C4  01  ; &¸.±.UÄ.&¸.±.UÄ.
00000060h:  26  B8  8D  B1  12  55  C4  01  26  B8  8D  B1  12  55  C4  01  ; &¸.±.UÄ.&¸.±.UÄ.
00000070h:  06  00  00  00  00  00  00  00  00  00  00  00  00  00  00  00  ; ................
00000080h:  00  00  00  00  00  01  00  00  00  00  00  00  00  00  00  00  ; ................
00000090h:  00  00  00  00  00  00  00  00  30  00  00  00  68  00  00  00  ; ........0...h...
```

NTFS Master File Table (MFT) Entry

Next 4 bytes is the attribute length ($len) in bytes

… **0x60** (96) bytes for this example

Attribute Header

```
00000000h:  46  49  4C  45  30  00  03  00  64  6F  11  01  00  00  00  00  ; FILE0...do......
00000010h:  01  00  01  00  38  00  01  00  A0  01  00  00  00  04  00  00  ; ....8...........
00000020h:  00  00  00  00  00  00  00  00  06  00  00  00  00  00  00  00  ; ................
00000030h:  50  00  00  00  00  00  00  00  10  00  00  00  60  00  00  00  ; P...........`...
00000040h:  00  00  18  00  00  00  00  00  48  00  00  00  18  00  00  00  ; ........H.......
00000050h:  26  B8  8D  B1  12  55  C4  01  26  B8  8D  B1  12  55  C4  01  ; &¸.±.UÄ.&¸.±.UÄ.
00000060h:  26  B8  8D  B1  12  55  C4  01  26  B8  8D  B1  12  55  C4  01  ; &¸.±.UÄ.&¸.±.UÄ.
00000070h:  06  00  00  00  00  00  00  00  00  00  00  00  00  00  00  00  ; ................
00000080h:  00  00  00  00  00  01  00  00  00  00  00  00  00  00  00  00  ; ................
00000090h:  00  00  00  00  00  00  00  00  30  00  00  00  68  00  00  00  ; ........0...h...
```

NTFS Master File Table (MFT) Entry

Next byte is the non-resident flag ($res)

0x00 – resident (attrib content in MFT entry)

0x01 – non-resident (content stored elsewhere)

Attribute Header

```
00000000h: 46 49 4C 45 30 00 03 00 64 6F 11 01 00 00 00 00 ; FILE0...do......
00000010h: 01 00 01 00 38 00 01 00 A0 01 00 00 00 04 00 00 ; ....8...........
00000020h: 00 00 00 00 00 00 00 00 06 00 00 00 00 00 00 00 ; ................
00000030h: 50 00 00 00 00 00 00 00 10 00 00 00 60 00 00 00 ; P...........`...
00000040h: 00 00 18 00 00 00 00 00 48 00 00 00 18 00 00 00 ; ........H.......
00000050h: 26 B8 8D B1 12 55 C4 01 26 B8 8D B1 12 55 C4 01 ; &¸.±.UÄ.&¸.±.UÄ.
00000060h: 26 B8 8D B1 12 55 C4 01 26 B8 8D B1 12 55 C4 01 ; &¸.±.UÄ.&¸.±.UÄ.
00000070h: 06 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ; ................
00000080h: 00 00 00 00 00 01 00 00 00 00 00 00 00 00 00 00 ; ................
00000090h: 00 00 00 00 00 00 00 00 30 00 00 00 68 00 00 00 ; ........0...h...
```

NTFS Master File Table (MFT) Entry

Next byte is the Length of allocated name of attribute

0x00- no name allocated for this example

Attribute Header

```
00000000h: 46 49 4C 45 30 00 03 00 64 6F 11 01 00 00 00 00 ; FILE0...do......
00000010h: 01 00 01 00 38 00 01 00 A0 01 00 00 00 04 00 00 ; ....8...........
00000020h: 00 00 00 00 00 00 00 00 06 00 00 00 00 00 00 00 ; ................
00000030h: 50 00 00 00 00 00 00 00 10 00 00 00 60 00 00 00 ; P...........`...
00000040h: 00 00 18 00 00 00 00 00 48 00 00 00 18 00 00 00 ; ........H.......
00000050h: 26 B8 8D B1 12 55 C4 01 26 B8 8D B1 12 55 C4 01 ; &¸.±.UÄ.&¸.±.UÄ.
00000060h: 26 B8 8D B1 12 55 C4 01 26 B8 8D B1 12 55 C4 01 ; &¸.±.UÄ.&¸.±.UÄ.
00000070h: 06 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ; ................
00000080h: 00 00 00 00 00 01 00 00 00 00 00 00 00 00 00 00 ; ................
00000090h: 00 00 00 00 00 00 00 00 30 00 00 00 68 00 00 00 ; ........0...h...
```

NTFS Master File Table (MFT) Entry

Next two bytes are the <u>name offset field</u>

0x00- no name allocated for this example

Attribute Header

```
00000000h: 46 49 4C 45 30 00 03 00 64 6F 11 01 00 00 00 00 ; FILE0...do......
00000010h: 01 00 01 00 38 00 01 00 A0 01 00 00 00 04 00 00 ; ....8...........
00000020h: 00 00 00 00 00 00 00 00 06 00 00 00 00 00 00 00 ; ................
00000030h: 50 00 00 00 00 00 00 00 10 00 00 00 60 00 00 00 ; P...........`...
00000040h: 00 00 18 00 00 00 00 00 48 00 00 00 18 00 00 00 ; ........H.......
00000050h: 26 B8 8D B1 12 55 C4 01 26 B8 8D B1 12 55 C4 01 ; &¸.±.UÄ.&¸.±.UÄ.
00000060h: 26 B8 8D B1 12 55 C4 01 26 B8 8D B1 12 55 C4 01 ; &¸.±.UÄ.&¸.±.UÄ.
00000070h: 06 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ; ................
00000080h: 00 00 00 00 00 01 00 00 00 00 00 00 00 00 00 00 ; ................
00000090h: 00 00 00 00 00 00 00 00 30 00 00 00 68 00 00 00 ; ........0...h...
```

NTFS Master File Table (MFT) Entry

Next two bytes are the status flags (only applies to the data of the attribute on the last block)

00 00 Normal

01 00 Compressed

40 00 Encrypted

80 00 Sparse

Attribute Header

```
00000000h: 46 49 4C 45 30 00 03 00 64 6F 11 01 00 00 00 00 ; FILE0...do......
00000010h: 01 00 01 00 38 00 01 00 A0 01 00 00 00 04 00 00 ; ....8...........
00000020h: 00 00 00 00 00 00 00 00 06 00 00 00 00 00 00 00 ; ................
00000030h: 50 00 00 00 00 00 00 00 10 00 00 00 60 00 00 00 ; P...........`...
00000040h: 00 00 18 00 00 00 00 00 48 00 00 00 18 00 00 00 ; ........H.......
00000050h: 26 B8 8D B1 12 55 C4 01 26 B8 8D B1 12 55 C4 01 ; &¸.±.UÄ.&¸.±.UÄ.
00000060h: 26 B8 8D B1 12 55 C4 01 26 B8 8D B1 12 55 C4 01 ; &¸.±.UÄ.&¸.±.UÄ.
00000070h: 06 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ; ................
00000080h: 00 00 00 00 00 01 00 00 00 00 00 00 00 00 00 00 ; ................
00000090h: 00 00 00 00 00 00 00 00 30 00 00 00 68 00 00 00 ; ........0...h...
```

NTFS Master File Table (MFT) Entry

Next two bytes are the Attribute ID

Attribute Header

```
00000000h: 46 49 4C 45 30 00 03 00 64 6F 11 01 00 00 00 00 ; FILE0...do......
00000010h: 01 00 01 00 38 00 01 00 A0 01 00 00 00 04 00 00 ; ....8...........
00000020h: 00 00 00 00 00 00 00 00 06 00 00 00 00 00 00 00 ; ................
00000030h: 50 00 00 00 00 00 00 00 10 00 00 00 60 00 00 00 ; P...........`...
00000040h: 00 00 18 00 00 00 00 00 48 00 00 00 18 00 00 00 ; ........H.......
00000050h: 26 B8 8D B1 12 55 C4 01 26 B8 8D B1 12 55 C4 01 ; &¸.±.UÄ.&¸.±.UÄ.
00000060h: 26 B8 8D B1 12 55 C4 01 26 B8 8D B1 12 55 C4 01 ; &¸.±.UÄ.&¸.±.UÄ.
00000070h: 06 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ; ................
00000080h: 00 00 00 00 00 01 00 00 00 00 00 00 00 00 00 00 ; ................
00000090h: 00 00 00 00 00 00 00 00 30 00 00 00 68 00 00 00 ; ........0...h...
```

NTFS Master File Table (MFT) Entry

Extra header fields for **resident** attributes

$sz_content – attribute content size (w/o header)	0x48 bytes
$ofs_content – attribute content offset	0x18 bytes

```
00000000h:  46  49  4C  45  30  00  03  00  64  6F  11  01  00  00  00  00  ; FILE0...do......
00000010h:  01  00  01  00  38  00  01  00  A0  01  00  00  00  04  00  00  ; ....8...........
00000020h:  00  00  00  00  00  00  00  00  06  00  00  00  00  00  00  00  ; ................
00000030h:  50  00  00  00  00  00  00  00  10  00  00  00  60  00  00  00  ; P...........`...
00000040h:  00  00  18  00  00  00  00  00  48  00  00  00  18  00  00  00  ; ........H.......
00000050h:  26  B8  8D  B1  12  55  C4  01  26  B8  8D  B1  12  55  C4  01  ; &¸.±.UÄ.&¸.±.UÄ.
00000060h:  26  B8  8D  B1  12  55  C4  01  26  B8  8D  B1  12  55  C4  01  ; &¸.±.UÄ.&¸.±.UÄ.
00000070h:  06  00  00  00  00  00  00  00  00  00  00  00  00  00  00  00  ; ................
00000080h:  00  00  00  00  00  01  00  00  00  00  00  00  00  00  00  00  ; ................
00000090h:  00  00  00  00  00  00  00  00  30  00  00  00  68  00  00  00  ; ........0...h...
```

Attribute Content

NTFS Master File Table (MFT) Entry

$STANDARD_INFORMATION

Offset	Size	Description
0x00	8	B Time – File Creation (Born)
0x08	8	M Time – File Modified
0x10	8	C Time – MFT Entry Changed
0x18	8	A Time – Last File Access time

Time Stamps

```
...
00000030h: 50 00 00 00 00 00 00 00 10 00 00 00 60 00 00 00 ; P...........`...
00000040h: 00 00 18 00 00 00 00 00 48 00 00 00 18 00 00 00 ; ........H.......
00000050h: 26 B8 8D B1 12 55 C4 01 26 B8 8D B1 12 55 C4 01 ; &¸.±.UÄ.&¸.±.UÄ.
00000060h: 26 B8 8D B1 12 55 C4 01 26 B8 8D B1 12 55 C4 01 ; &¸.±.UÄ.&¸.±.UÄ.
00000070h: 06 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ; ................
00000080h: 00 00 00 00 00 01 00 00 00 00 00 00 00 00 00 00 ; ................
00000090h: 00 00 00 00 00 00 00 00 30 00 00 00 68 00 00 00 ; ........0...h...
```

NTFS Master File Table (MFT) Entry

Content format for the **$FILE_NAME** attribute

Offset	Size	Description
0x00	8	File Reference to parent directory
0x08	8	B Time - File Creation (Born)
0x10	8	M Time – File Modify
0x18	8	C Time - MFT Entry Changed
0x20	8	A Time – File Access
0x28	8	Allocated size of file
0x30	8	Real size of file
0x40	1	Filename length in unicode characters
0x41	1	Filename namespace
0x42		File Name in Unicode

NTFS Master File Table (MFT) Entry

Content format for the **$FILE_NAME** attribute

- Note: Some files may contain 2 $FILE_NAME fields
  - 8-char - for compatibility with older OS
  - Multi-char – for newer OS
- MACB values stored in this attribute are usually not affected by system activity
  - Can be used to detect tampering of file timestamps

NTFS Master File Table (MFT) Entry

Getting the MACB values:

- Time denoted in UTC
- Useful for reconstructing a timeline by generating the *fls* tool *bodyfile*

```
fls -f ntfs -o <offset> -r -m -p
<image> > <output file>
```

NTFS Master File Table (MFT) Entry

The **$DATA** attribute:

- Points to the content of the file
- **Location** of file content depends on the $res (non-resident flag) attribute header field

  $res =00x00

- attribute content includes the **file data**
- usually done for small files that fit in an MFT entry
- Use $sz_content and $ofs_content to retrieve

  $res = 0x01

- attribute content is **a pointer** to the file data run
  - sectors where actual file data is stored

NTFS Alternate Data Streams

- NTFS stores files as a series of attributes, such as the name of the file, or a timestamp
- The data which the file contains is stored in the attribute **$DATA**, and is known as a **data stream**
- By using NTFS, Alternate Data Streams (ADSs) can be connected to the file
- An attacker could store malicious code within an ADS that can then be called from a different file
- In the NTFS file system, a file with an ADS is identified after the filename and a colon, for example, **Testfile.txt:ADS**. This filename indicates an ADS called ADS is associated with the file called **Testfile.txt**

```
C:\ADS> echo "Alternate Data Here" > Testfile.txt:ADS
C:\ADS> dir
 Volume in drive C is Windows
 Volume Serial Number is A606-CB1B
 Directory of C:\ADS
2020-04-28  04:01 PM    <DIR>          .
2020-04-28  04:01 PM    <DIR>          ..
2020-04-28  04:01 PM                 0 Testfile.txt
               1 File(s)              0 bytes
               2 Dir(s)  43,509,571,584 bytes free
C:\ADS> more < Testfile.txt:ADS
"Alternate Data Here"
C:\ADS> dir /r
 Volume in drive C is Windows
 Volume Serial Number is A606-CB1B
 Directory of C:\ADS
2020-04-28  04:01 PM    <DIR>          .
2020-04-28  04:01 PM    <DIR>          ..
2020-04-28  04:01 PM                 0 Testfile.txt
                                    24 Testfile.txt:ADS:$DATA
               1 File(s)              0 bytes
               2 Dir(s)  43,509,624,832 bytes free
C:\ADS>
```

Windows Operations

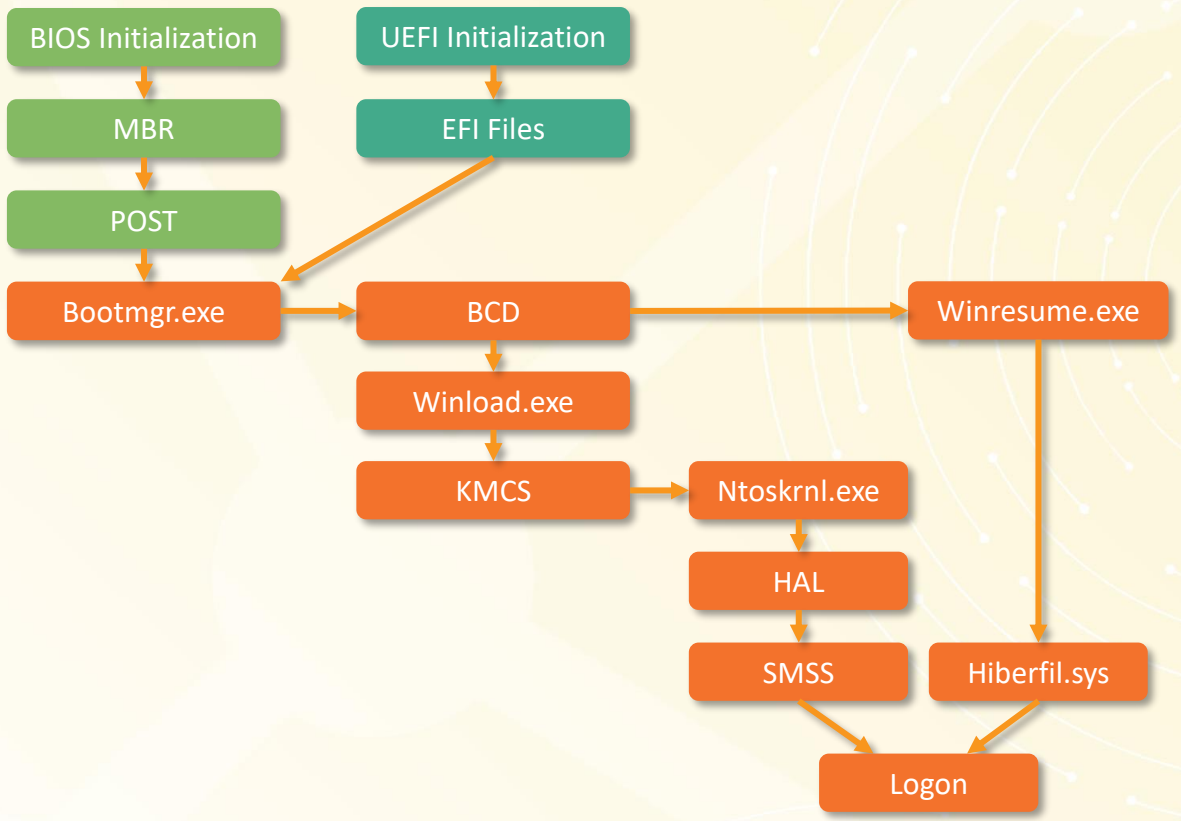
Boot Process | Startup | Shutdown | Processes, Threads, Services

Windows Boot Process

- Many actions occur between the power button is pressed and Windows is fully loaded. This is the Windows Boot process

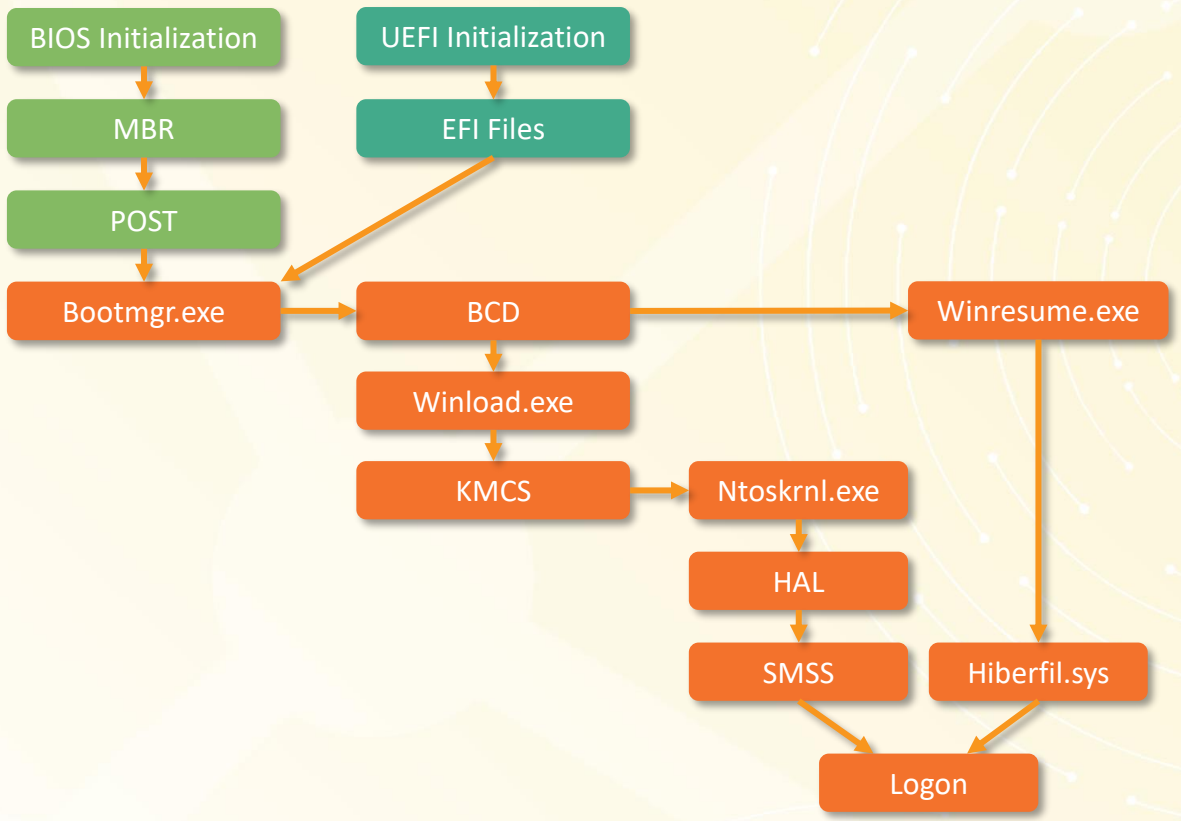
Two types of computer firmware exist:

- **Basic Input-Output System (BIOS):** The process begins with the BIOS initialization phase in which the hardware devices are initialized and a **POST (Power-On Self-Test)** is performed. When the system disk is discovered, the POST ends and looks for the Master Boot Record (MBR). The BIOS executes the MBR code and the operating systems starts to load
- **Unified Extensible Firmware Interface (UEFI):** UEFI firmware boots by loading EFI program files (.efi) stored in a special disk partition, known as the EFI System Partition (ESP)



Windows Boot Process

- Whether the firmware is **BIOS or UEFI**, after a valid Windows installation is located, the **Bootmgr.exe file** is run
- Bootmgr.exe reads the **Boot Configuration Database (BCD)**
- If the computer is coming out of hibernation, the boot process continues with **Winresume.exe**
- If the computer is being booted from a cold start, then the **Winload.exe** file is loaded
- Winload.exe also uses **Kernel Mode Code Signing (KMCS)** to make sure that all drivers are digitally signed
- After the drivers have been examined, Winload.exe runs **Ntoskrnl.exe** that starts the Windows kernel and sets up the **HAL**.



Windows Startup

- There are two important registry items that are used to automatically start applications and services:
  - **HKEY_LOCAL_MACHINE**
    - Several aspects of Windows configuration are stored in this key, including information about services that start with each boot
  - **HKEY_CURRENT_USER**
    - Several aspects related to the logged in user are stored in this key, including information about services that start only when the user logs on to the computer
- Different entries in these registry locations define which services and applications will start, as indicated by their entry type

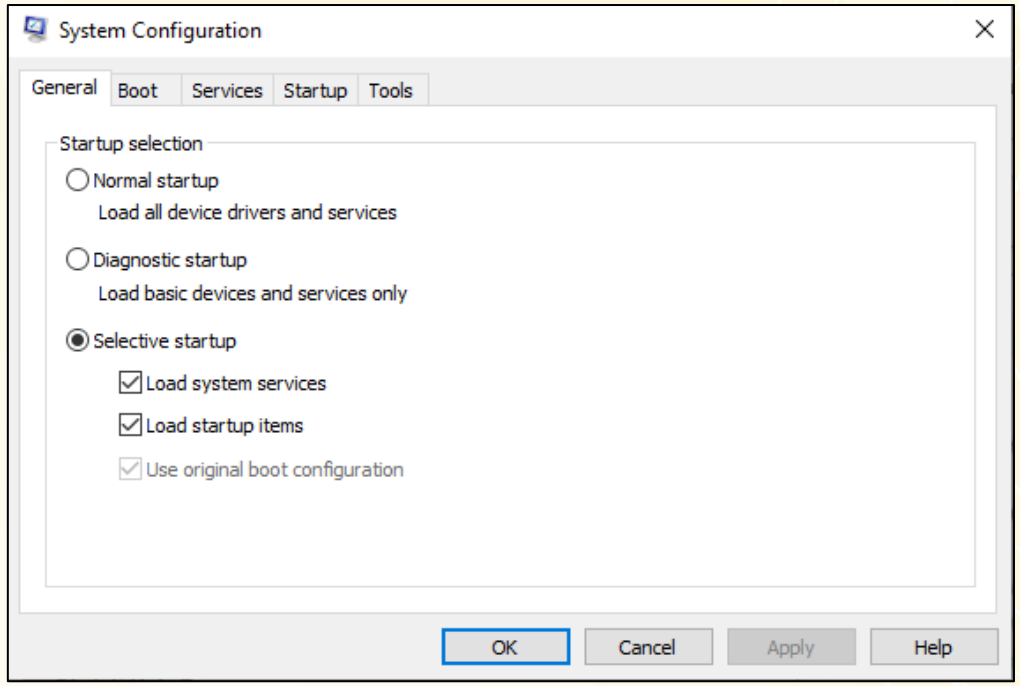
Windows Startup

- These types include Run, RunOnce, RunServices, RunServicesOnce, and UserInit. These entries can be manually entered into the registry, but it is much safer to use the **Msconfig.exe** tool

- The **msconfig tool** is used to <u>view and change all the start-up options</u> for the computer. It opens the System Configuration window

Windows Startup

There are five tabs that contain the configuration options



General

Three different startup types can be chosen here:

- **Normal**
  - Loads all drivers and services
- **Diagnostic**
  - Loads only basic drivers and services
- **Selective**
  - Allows the user to choose what to load on startup

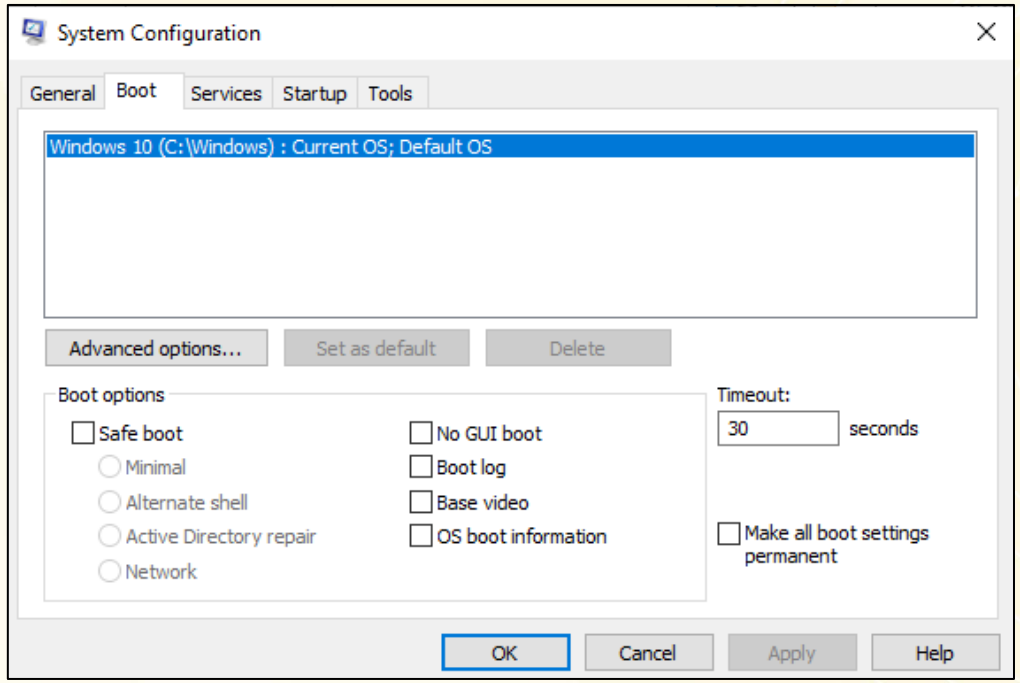
Windows Startup

There are five tabs that contain the configuration options

Boot

- Any installed operating system can be chosen here to start

- There are also options for Safe boot, which is used to troubleshoot startup

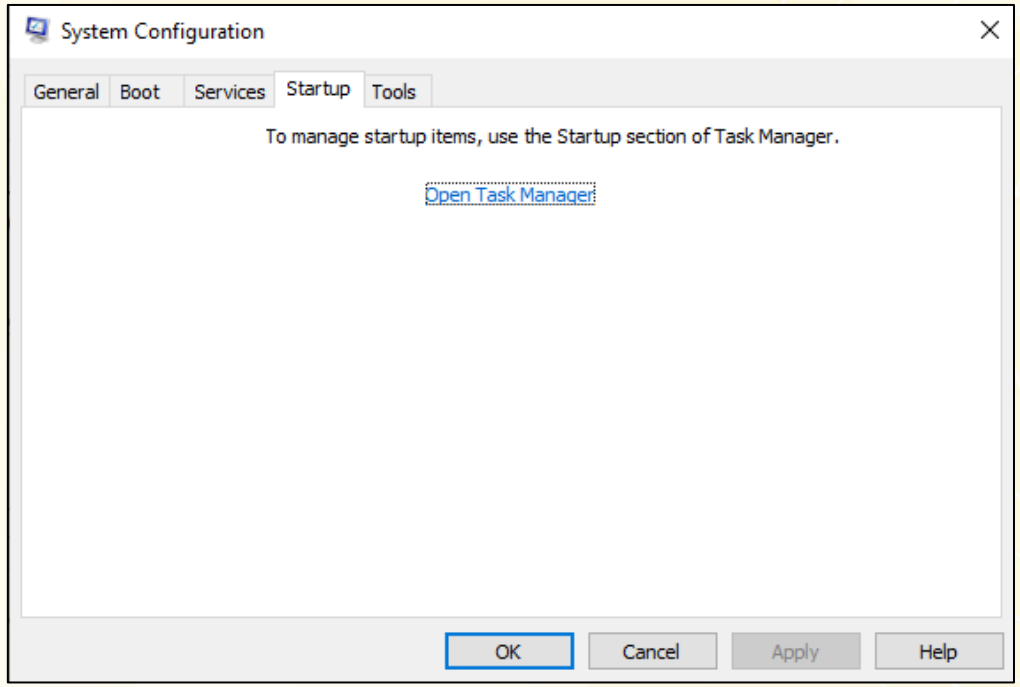


Windows Startup

There are five tabs that contain the configuration options

Startup

- All the applications and services that are configured to automatically begin at startup can be enabled or disabled by opening the task manager from this tab

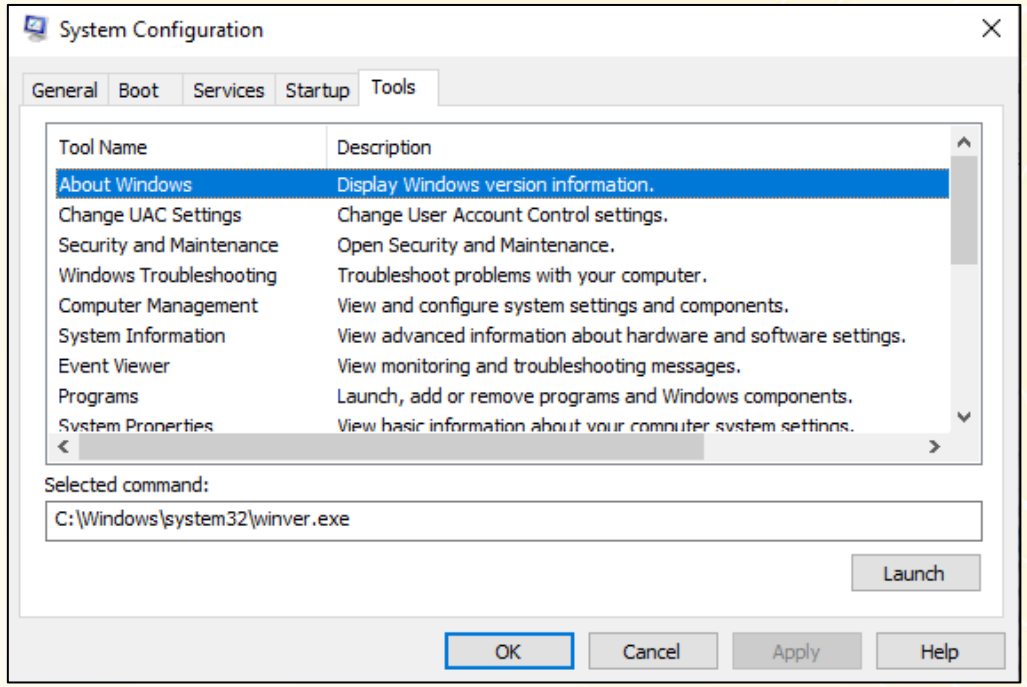


Windows Startup

There are five tabs that contain the configuration options

Tools

- May common operating system tools can be launched directly from this tab



Windows Shutdown

- It is always best to perform a proper shutdown to turn off the computer. The computer needs time to close each application, shut down each service, and record any configuration changes before power is lost
- During shutdown, the computer will close user mode applications first, followed by kernel mode processes
- There are several ways to shutdown a Windows computer:
  - Start menu power options
  - Command line command **shutdown**
  - Using **Ctrl+Alt+Delete** and clicking the power icon

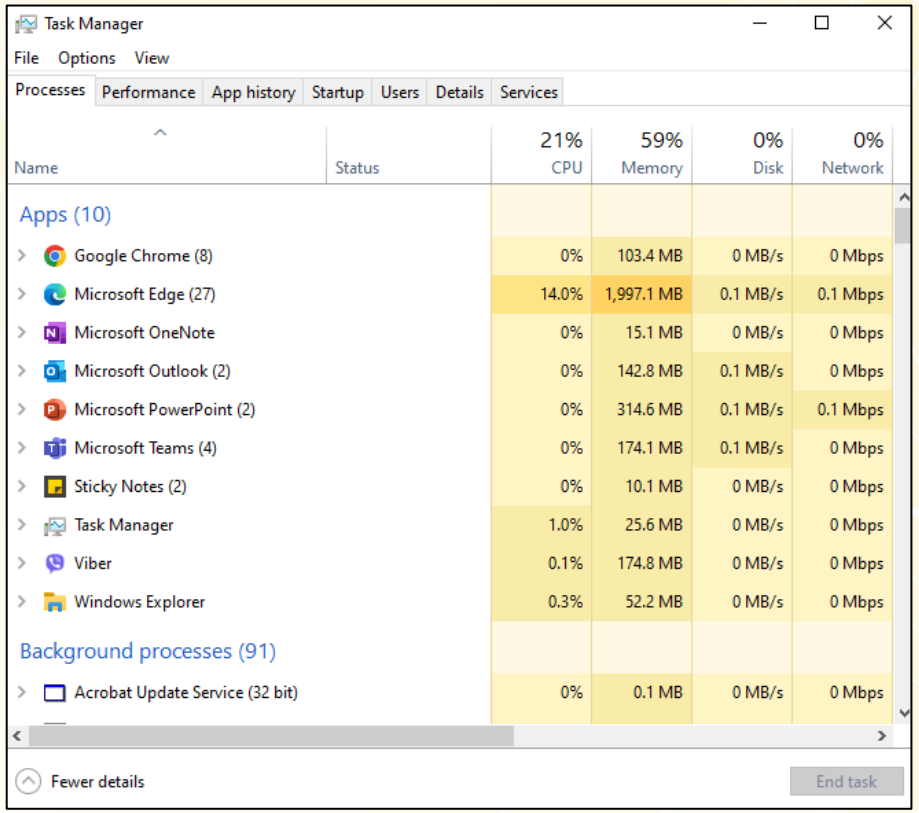
Windows Shutdown

- There are three different options from which to choose when shutting down the computer:
- **Shutdown:** Turns the computer off (power off)
- **Restart:** Reboots the computer (power off and power on)
- **Hibernate:** Records the current state of the computer and user environment and stores it in a file. Hibernation allows the user to pick up right where they left off very quickly with all their files and programs still open

Processes, Threads, and Services

- A Windows application is made up of processes. A process is any program that is currently executing

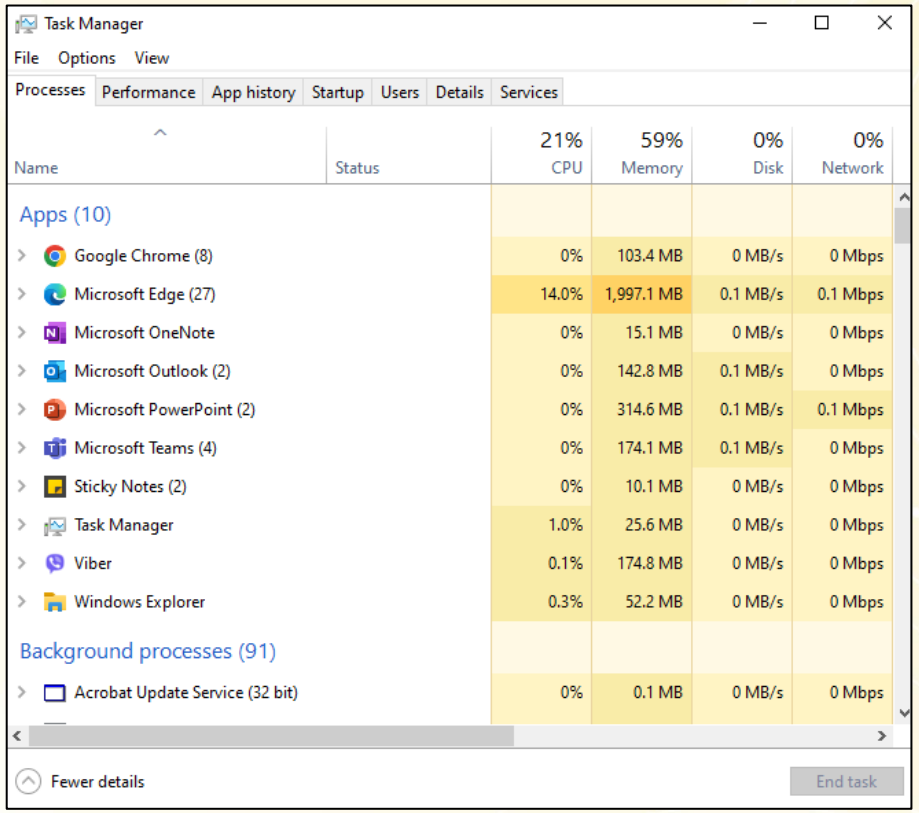
- Each process that runs is made up of at least one thread. A thread is a part of the process that can be executed



Processes, Threads, and Services

- To configure Windows processes, search for Task Manager. The Processes tab of the Task Manager is shown in the figure

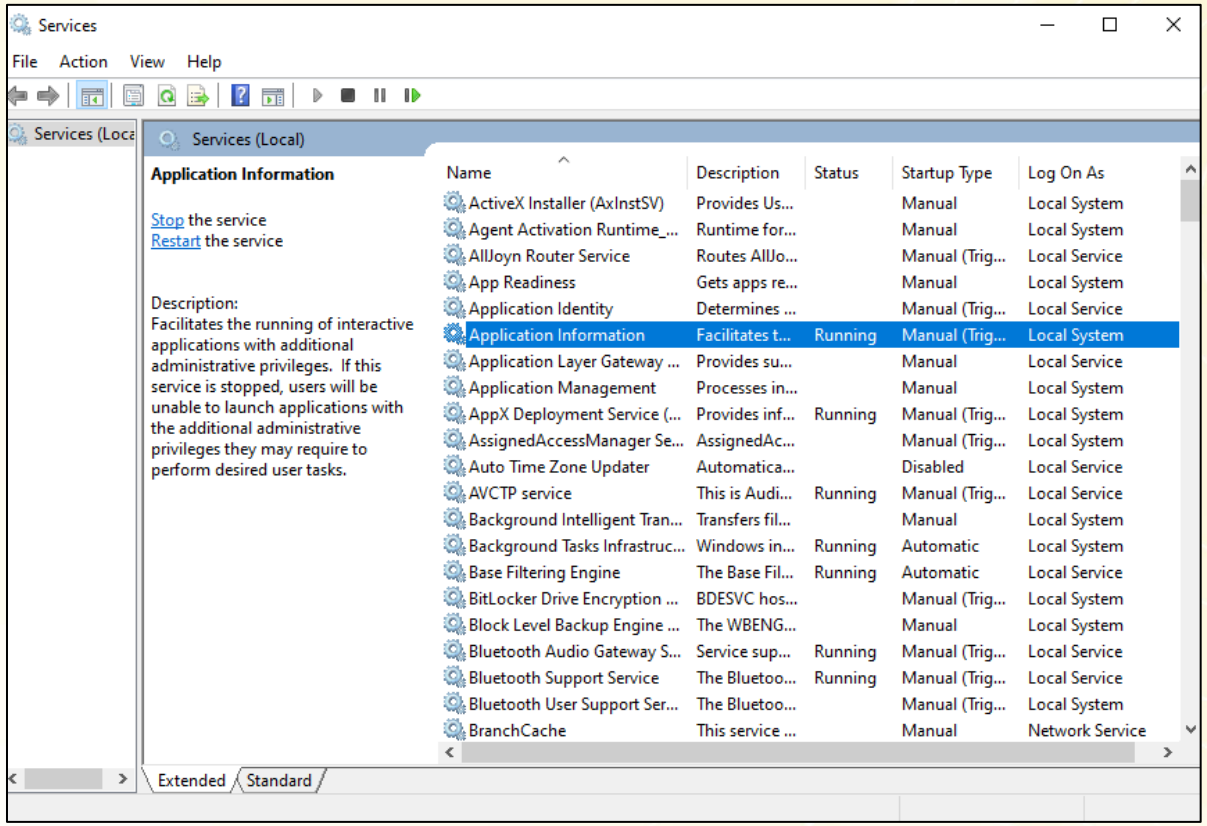
- All of the threads dedicated to a process are contained within the same address space which means that these threads may not access the address space of any other process. This prevents corruption of other processes



Processes, Threads, and Services

- Some of the processes that Windows runs are services. These are programs that run in the background to support the operating system and Applications

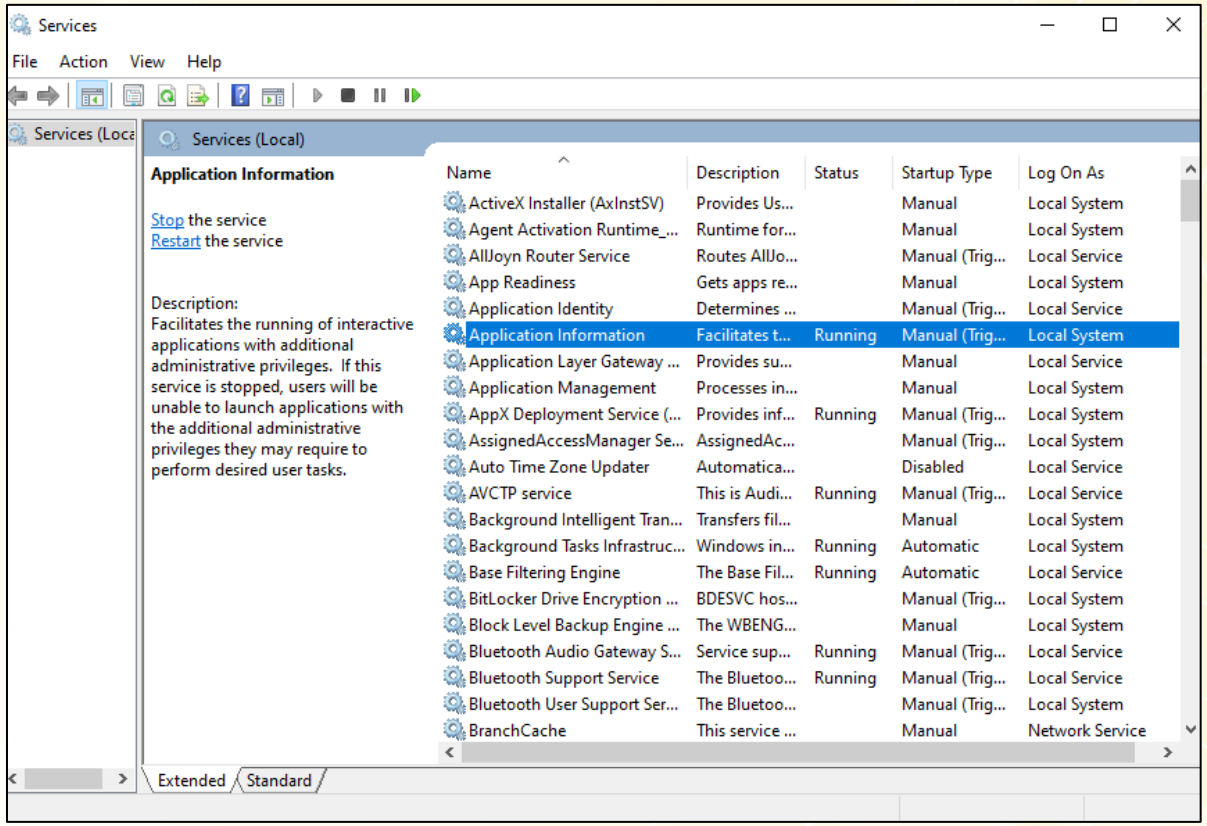
- Service provide long-running functionality, such as wireless or access to an FTP Server



Processes, Threads, and Services

- To configure Windows Services,earch for services. The Windows Services control panel applet is shown in the figure

- Be very careful when manipulating the settings of these services. Shutting down a service may adversely affect applications or services



Memory Allocation and Handles

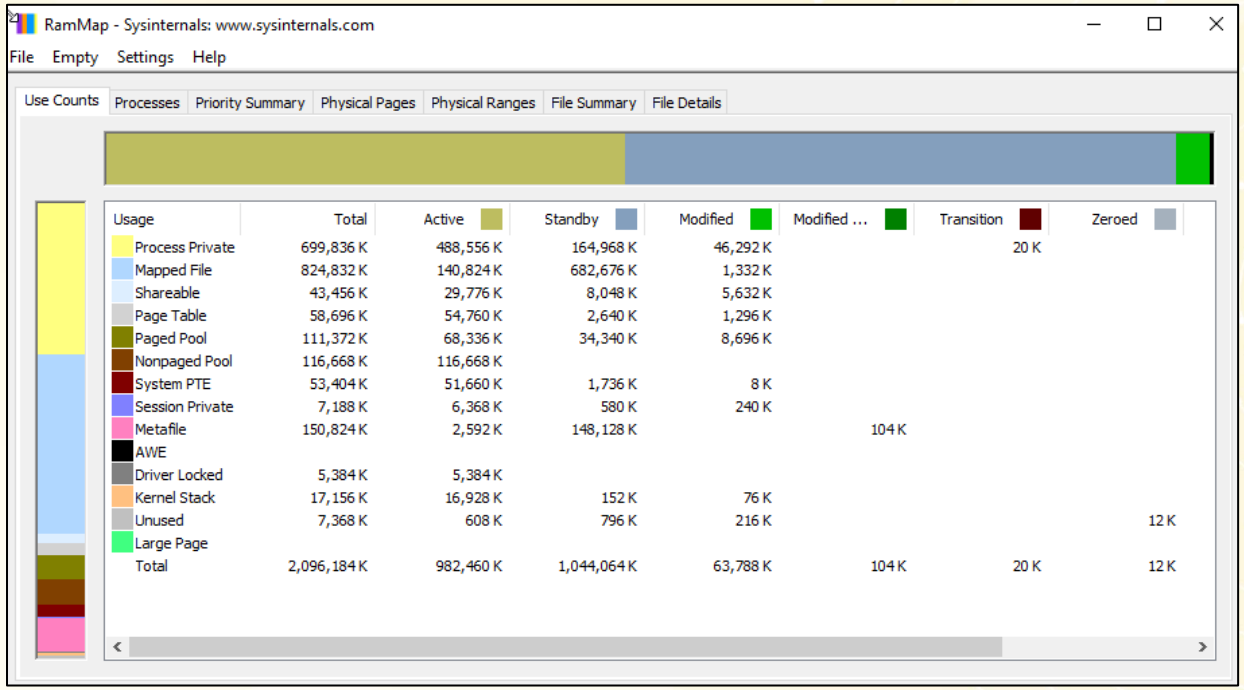
- The virtual address space for a process is the set of virtual addresses that the process can use
- The virtual address is not the actual physical location in memory, but an entry in a page table that is used to translate the virtual address into the physical address
- Each process in a 32-bit Windows computer supports a virtual address space that enables addressing up to 4 gigabytes
- Each process in a 64-bit Windows computer supports a virtual address space of 16 terabytes

Memory Allocation and Handles

- Each user space process runs in a private address space, separate from other user space processes
- When the user space process needs to access kernel resources, it must use a process handle
- As the user space process is not allowed to directly access these kernel resources, the process handle provides the access needed by the user space process without a direct connection to it

Processes, Threads, and Services

- A powerful tool for viewing memory allocation is **RAMMap**, which is shown in the figure



Usage	Total	Active	Standby	Modified	Modified ...	Transition	Zeroed
Process Private	699,836 K	488,556 K	164,968 K	46,292 K		20 K	
Mapped File	824,832 K	140,824 K	682,676 K	1,332 K			
Shareable	43,456 K	29,776 K	8,048 K	5,632 K			
Page Table	58,696 K	54,760 K	2,640 K	1,296 K			
Paged Pool	111,372 K	68,336 K	34,340 K	8,696 K			
Nonpaged Pool	116,668 K	116,668 K					
System PTE	53,404 K	51,660 K	1,736 K	8 K			
Session Private	7,188 K	6,368 K	580 K	240 K			
Metafile	150,824 K	2,592 K	148,128 K		104 K		
AWE							
Driver Locked	5,384 K	5,384 K					
Kernel Stack	17,156 K	16,928 K	152 K	76 K			
Unused	7,368 K	608 K	796 K	216 K			12 K
Large Page							
Total	2,096,184 K	982,460 K	1,044,064 K	63,788 K	104 K	20 K	12 K

- RAMMap is part of the Windows Sysinternals Suite of tools. It can be downloaded from Microsoft

- RAMMap provides information regarding how Windows has allocated system memory to the kernel, processes, drivers, and applications

Windows Artifacts

SANS FOR500: Windows Artifacts Analysis: Evidence of…

Windows Artifacts Categories (WAC)

- Application Execution
- Browser Activity
- Cloud Storage
- Network Activity and Physical Location
- File and Folder Opening
- Account Usage
- Deleted Items and File Existence
- External Device / USB Usage
- System Information

Windows Artifacts Analysis:

- Evidence of…
  1. Application Execution (AppExec)
  2. Browser Activity (BrowAct)
  3. Cloud Storage (CloSto)
  4. Network Activity and Physical Location (NAPL)
  5. File and Folder Opening (FiFOp)
  6. Account Usage (AccUs)
  7. Deleted Items and File Existence (DIFE)
  8. External Device / USB Usage (ExDev)
  9. System Information (SysInfo)

Windows Registry

Queen P and their Hives

Windows Registry

- Windows stores all of the information about hardware, applications, users, and system settings in a large database known as **the registry**
- The registry is a hierarchical database where the highest level is known as a hive, below that there are keys, followed by subkeys
- Composed of multiple files (hives) mostly stored in the system32 folder
- Values store data and are stored in the keys and subkeys. A registry key can be up to 512 levels deep

Windows Registry

- The following table lists the five hives of the Windows registry:

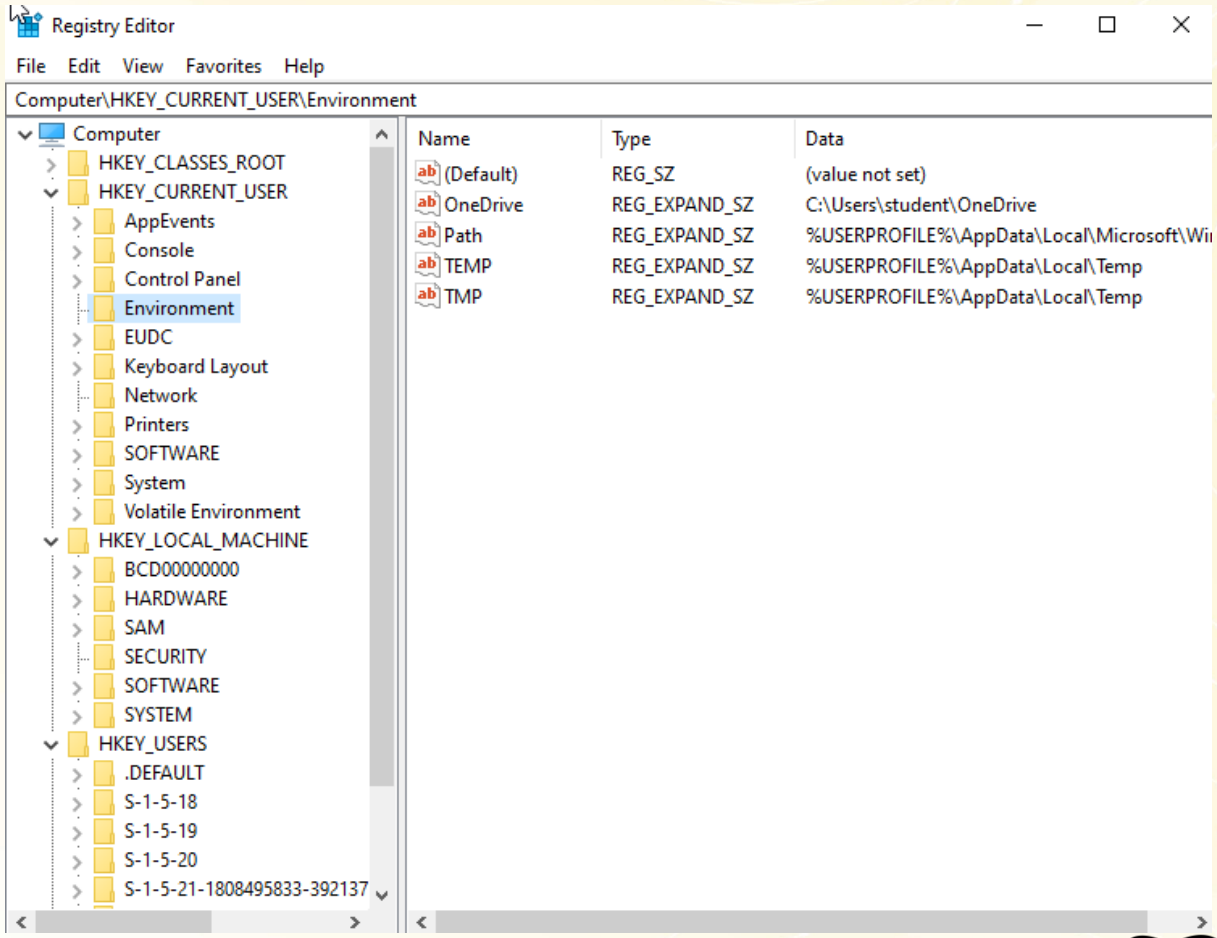
Registry Hive	Description
HKEY_CURRENT_USER (HKCU)	Holds information concerning the currently logged in user
HKEY_USERS (HKU)	Holds information concerning all the user accounts on the host
HKEY_CLASSES_ROOT (HKCR)	Holds information about Object linking and embedding (OLE) registrations. It allows users to embed objects from other applications a single document
HKEY_LOCAL_MACHINE (HKLM)	Holds system-related information.
HKEY_CURRENT_CONFIG (HKCC)	Holds the current hardware profile

Windows Registry

- **SAM**
  - Contains system user group and account information
- **Security**
  - Contains user access policies
- **Software**
  - Contains information specific to installed software (e.g., configs, file type associations)
- **System**
  - Contains system setup information (e.g., hardware and driver configs)
- **NTUser.dat**
  - Contains settings specific to a user account

Windows Registry

- New hives cannot be created. The registry keys and values in the hives can be created, modified, or deleted by an account with administrative Privileges
- As shown in the figure, the tool **regedit.exe** is used to modify the Registry
- Be very careful when using this tool. Minor changes to the registry can have massive or even catastrophic effects

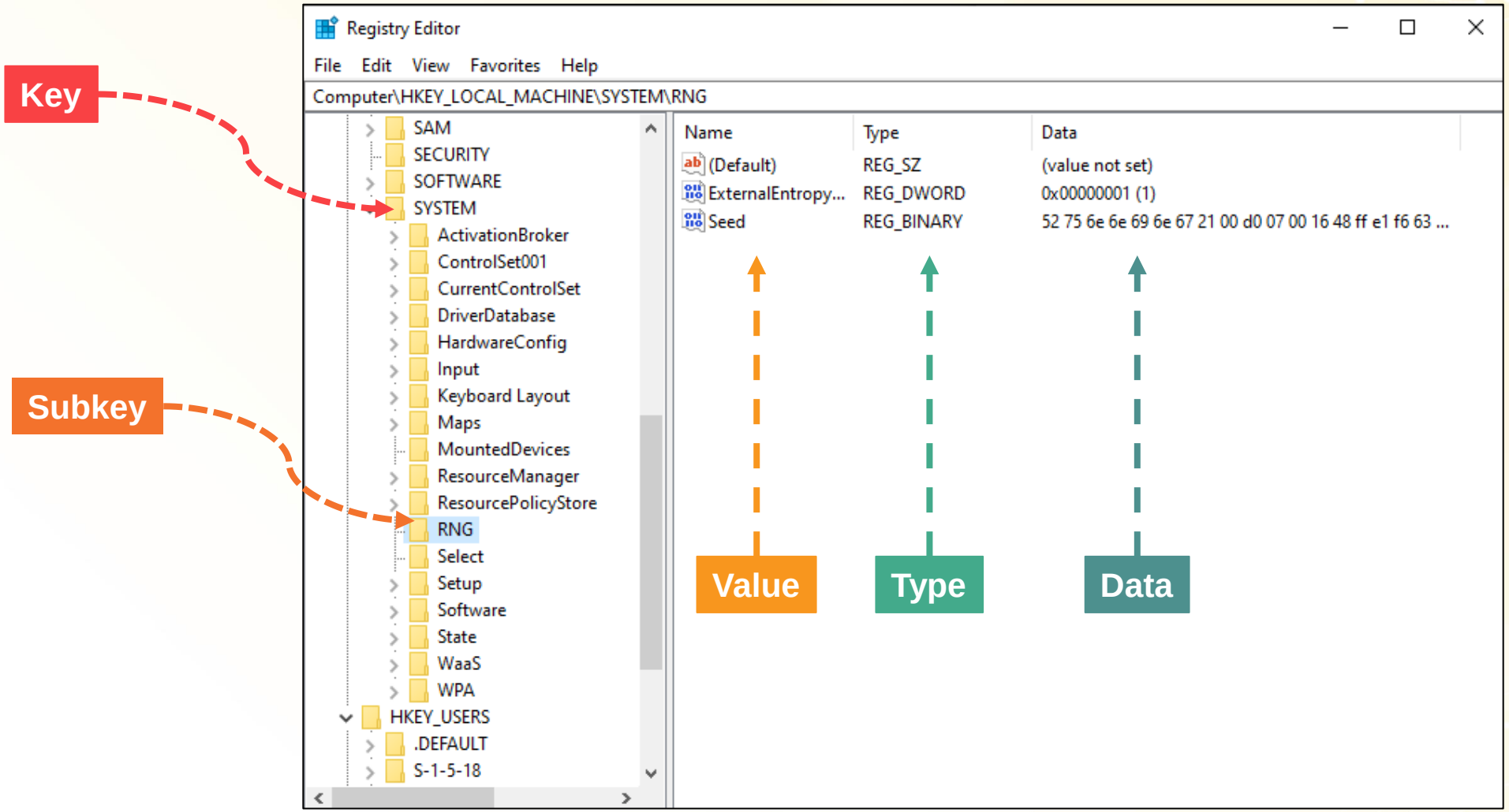


Windows Registry

Binary structure consists of “cells”: keys and values

- **Keys**
  - Container objects
  - Work similar to folders
  - May contain values or subkeys
- **Values**
  - Non-container objects
  - Contain a name and corresponding value/data

Windows Registry



Key Cell Structure

```
A0 FF FF FF 6E 6B 2C 00 82 4D EF 2C 8F 55 C4 01
00 00 00 00 F8 01 00 00 07 00 00 00 01 00 00 00
B8 E4 0F 00 78 01 00 80 00 00 00 00 FF FF FF FF
80 00 00 00 FF FF FF FF 2A 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
24 24 24 50 52 4F 54 4F 2E 48 49 56 00 00 00 00
```

Hive File Contents

Offset (bytes)	Size (bytes)	Description	Value
0	4	Size	0xFFFFFFA0
4	2	Node ID (“nk”)	0x6B6E
6	2	Node Type (0x2C or 0x20)	0x002C
8	8	Last Write time	0x01C4558F2CEF4D82
20	4	Offset to this key’s parent key	0x000001F8
24	4	Number of subkeys	0x00000007
32	4	Offset to the list of subkey records	0x000FE4B8
36	4	Number of values	0x80000178
44	4	Offset to the value list	0xFFFFFFFF
48	4	Offset to security identifier record	0X00000000
76	2	Length of the key name	

Value Structure

```
D8 FF FF FF 76 6B 0B 00 12 00 00 00 68 03 00 00
01 00 00 00 01 00 3D E1 43 75 72 72 65 6E 74 55
73 65 72 00 00 00 00 00 E8 FF FF FF 55 00 53 00
45 00 52 00 4E 00 41 00 4D 00 45 00 00 00 3D E1
```

Hive File Contents

Offset (bytes)	Size (bytes)	Description	Value
0	4	Size (as a negative number)	0XFFFFFFD8
4	2	Node ID (“vk,” or 0x6B76)	0x6B76
6	2	Value name length	0x000B
8	4	Data length	0x00000012
12	4	Offset to data	0x00000368
16	4	Value type	0x00000001

Hive Files

- Navigation in the registry is very similar to the Windows File Explorer
- Use the left panel to navigate the hives and the structure below it and use the right panel to see the contents of the highlighted item in the left panel
- The path is displayed at the bottom of the windows for reference
- Registry keys can contain either a subkey or a value. The different values that keys can contain are as follows:
  - **REG_BINARY**: Numbers or Boolean values
  - **REG_DWORD**: Numbers greater than 32 bits or raw data
  - **REG_SZ**: String values

Hive Files

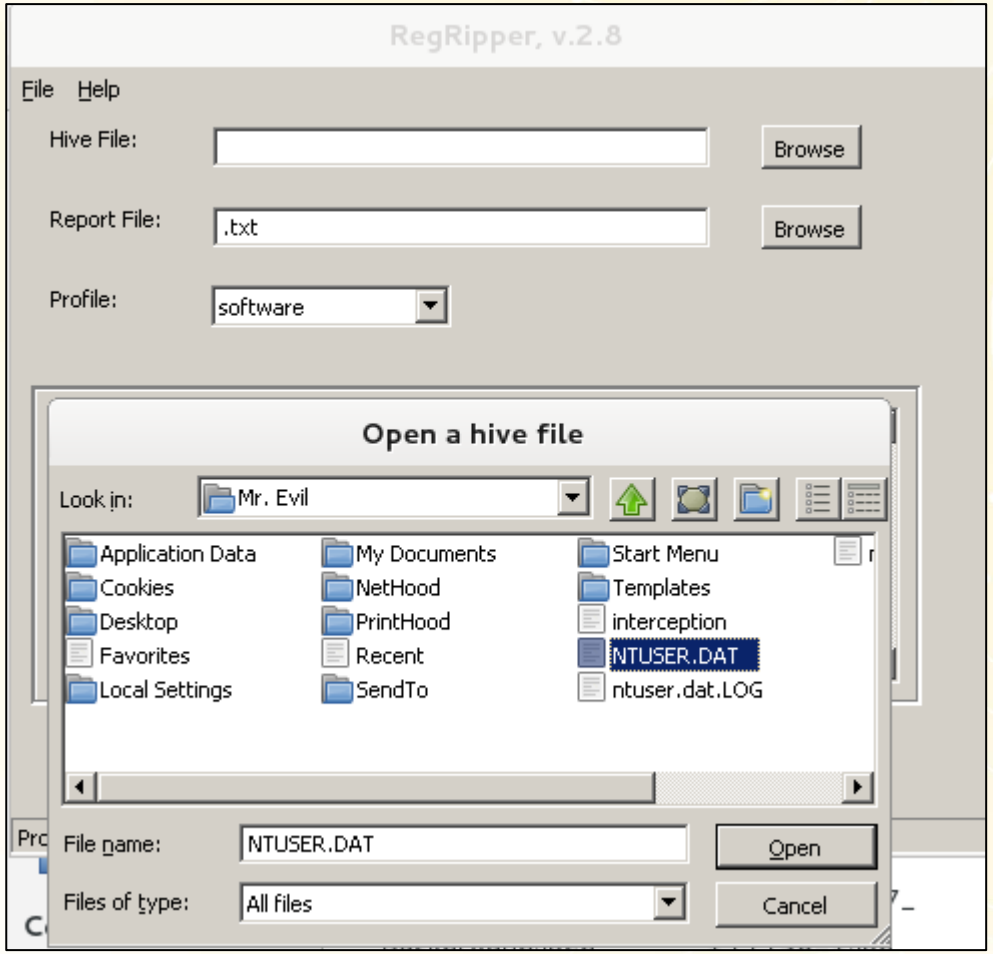
- The registry also contains the activity that a user performs during normal day-to-day computer use

- This includes the history of hardware devices, including all devices that have been connected to the computer including the name, manufacturer, and serial number

Hive Files

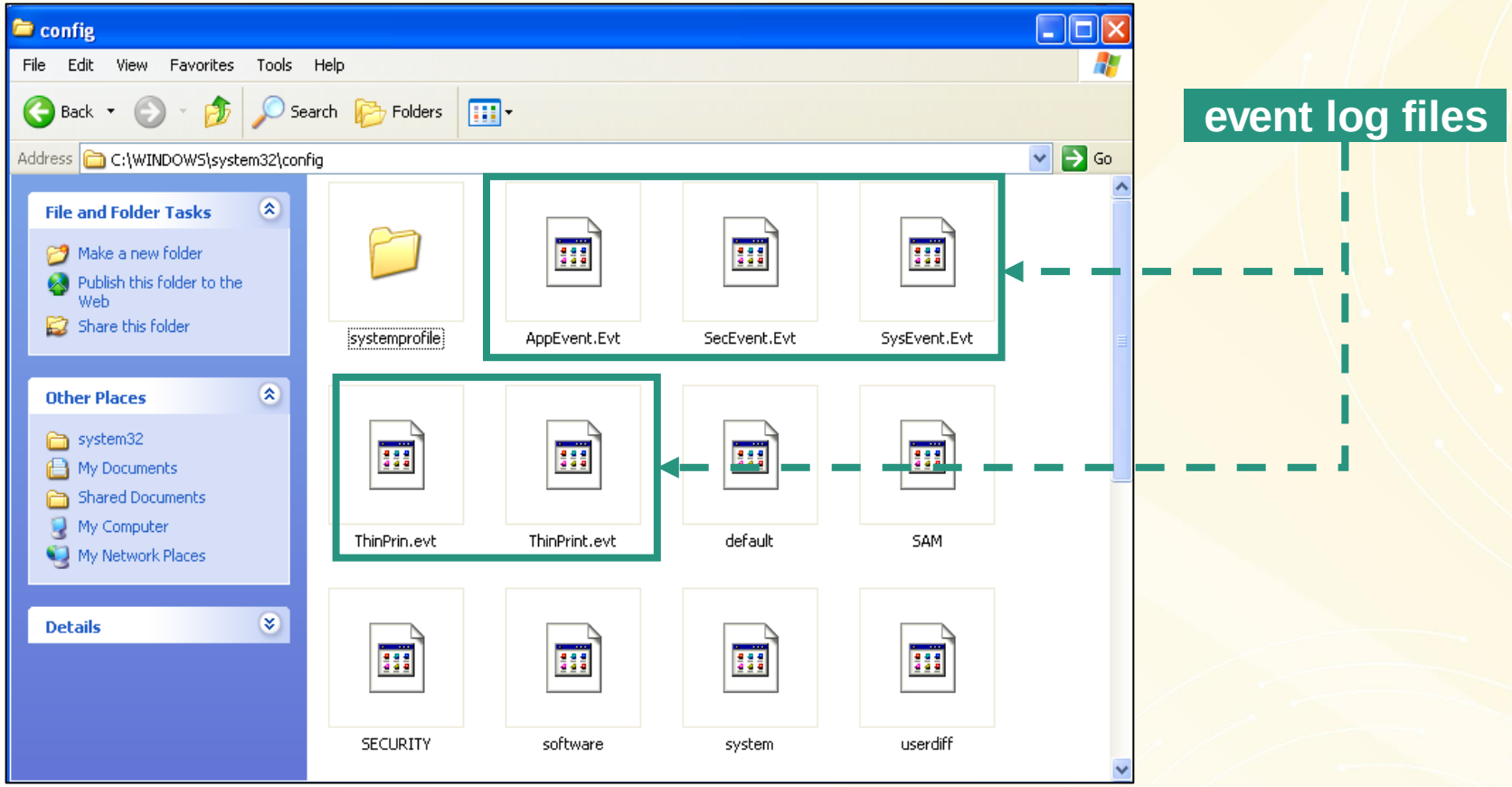
RegRipper

- An open-source Perl application available for parsing Registry Hives



Event Logs

Logging System of Windows



Event Logs

Event Log File Header

Event Log Record Header

00h:	30	00	00	00	4C	66	4C	65	01	00	00	00	01	00	00	00
10h:	30	00	00	00	7C	51	00	00	6B	00	00	00	01	00	00	00
20h:	00	00	01	00	08	00	00	00	00	00	00	00	30	00	00	00
30h:	E0	00	00	00	4C	66	4C	65	01	00	00	00	4F	B0	D2	40
40h:	6D	B0	D2	40	02	00	06	40	04	00	02	00	00	00	6F	00
50h:	73	00	00	00	5E	00	00	00	00	00	00	00	00	00	00	00
60h:	30	00	00	00	9E	00	00	00	53	00	65	00	72	00	69	00
70h:	61	00	6C	00	00	00	4D	00	41	00	43	00	48	00	49	00

Event Logs

bracket (0x00000030)

signature (LfLe)

bracket (0x00000030)

Event Log File Header

Event Log Record Header

00h:	30	00	00	00	4C	66	4C	65	01	00	00	00	01	00	00	00
10h:	30	00	00	00	7C	51	00	00	6B	00	00	00	01	00	00	00
20h:	00	00	01	00	08	00	00	00	00	00	00	00	30	00	00	00
30h:	E0	00	00	00	4C	66	4C	65	01	00	00	00	4F	B0	D2	40
40h:	6D	B0	D2	40	02	00	06	40	04	00	02	00	00	00	6F	00
50h:	73	00	00	00	5E	00	00	00	00	00	00	00	00	00	00	00
60h:	30	00	00	00	9E	00	00	00	53	00	65	00	72	00	69	00
70h:	61	00	6C	00	00	00	4D	00	41	00	43	00	48	00	49	00

Size (0x000000E0 or 224)

signature (LfLe)

time written

Difference?

time generated

Event Logs

evtrpt.pl

Extracts and displays the event record information keeps track of that information and displays statistics about the frequency of various event identifiers (IDs) and sources for the event records

```
evtrpt e:\WINDOWS\system32\config\S
ys Event.Evt
```

```
-------------------------------
Event Source/ID Frequency

Source                          Event ID    Count
----------                      --------    -----
DCOM                            10005         4
Dhcp                             1005         1
:
:
Service Control Manager          7036        36
Setup                           60054         1
Setup                           60055         1
W32Time                            35         3

Total: 106
-------------------------------
Event Type Frequency

Type                     Count
----------               -----
Error                      9
Info                      91
Warn                       6

Total: 106
-------------------------------
Date Range (UTC)
Fri Jun 18 09:05:19 2004 to Fri Jan 18 00:53:41 2008
```

Prefetch Files

Prefetching

Allows regularly used applications to load faster pre-stages segments of loaded code in a specific location instead of searching for it (resulting in page faults), the operating system would know exactly where to find that code. When an application is launched, a **prefetch file** for that application is created

70h:	01	00	00	00	64	04	00	00	**82**	**25**	**0C**	**68**	**16**	**6F**	**CB**	**01**
80h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
90h:	**03**	**00**	**00**	**00**	01	00	00	00	00	00	00	00	00	00	00	00

Time last run

Run count

Prefetch Files

pref.pl

retrieve these metadata from prefetch files from Windows XP, as well as from Vista and Windows 7 systems

```
pref -f     e:\WINDOWS\Prefetch\<prefetch file name>          or
pref -d     e:\WINDOWS\Prefetch

e:\WINDOWS\Prefetch\AGENTSVR.EXE-002E45AB.pf Fri Jun 18 18:58:33 2004 (1)
e:\WINDOWS\Prefetch\CACLS.EXE-25504E4A.pf Fri Jun 18 23:49:03 2004 (2)
e:\WINDOWS\Prefetch\CMD.EXE-087B4001.pf Fri Jan 18 00:52:42 2008 (3)
e:\WINDOWS\Prefetch\DEFRAG.EXE-273F131E.pf Fri Jun 18 19:15:51 2004 (1)
e:\WINDOWS\Prefetch\DFRGNTFS.EXE-269967DF.pf Fri Jun 18 19:15:52 2004 (1)
e:\WINDOWS\Prefetch\EXPLORER.EXE-082F38A9.pf Fri Jun 18 18:59:36 2004 (1)
```

Prefetch Files

Aside from containing run count and time last run, prefetch files also contains:

Volume information

```
Volume path: \DEVICE\HARDDISKVOLUME1
Volume Creation Date: Fri Jan 1 22:24:09
Volume Serial Number: A424-CE42
```

Other file path

```
\DEVICE\HARDDISKVOLUME1\ADS\MYFILE.TXT: ADS.TXT
```

Windows Executables

Executable files on Windows systems

- follows the **portable executable (PE), common object file format (COFF)** specification
- under normal circumstances, files with specific extensions (i.e., **.exe, .dll, .sys, .ocx**) will follow a particular **binary structure format**

Windows Executables

‘MZ’ Signature or DOS Header / IMAGE_DOS_HEADER / Magic Number 0x5A4D or “ZM”

00h:	4D	5A	90	00	03	00	00	00	04	00	00	00	FF	FF	00	00
10h:	B8	00	00	00	00	00	00	00	40	00	00	00	00	00	00	00
20h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
30h:	00	00	00	00	00	00	00	00	00	00	00	00	E0	00	00	00
.																
.																
.																
E0h:	50	45	00	00	4C	01	03	00	87	52	02	48	00	00	00	00

Offset to PE Header

‘PE’ Signature / IMAGE_NT_HEADER 0x4550 or “EP”

Windows Executables

pescanner.py

retrieve metadata within a Windows Executable file

```
python pescanner.py e:\windows\notepad.exe

Meta-data
=================================================================
File: f:\WINDOWS\NOTEPAD.EXE
Size: 66048 bytes
:
:
FileVersion: 5.1.2600.0 (xpclient.010817-1148)
CompanyName: Microsoft Corporation
ProductName: Microsoft\xae Windows\xae Operating System
ProductVersion: 5.1.2600.0
FileDescription: Notepad
OriginalFilename: NOTEPAD.EXE
```